🧪 GUÍA DE LABORATORIO

Representación de Pose y Cuaterniones en ROS2 (Capítulo 2 – Corke)

📌 Información general

- **Curso:** Robótica
- **Modalidad:** Práctica con simulación (Python + ROS2)
- **Herramientas:** ROS2, TF2, RViz, SciPy

🎯 Objetivos de aprendizaje

Al finalizar el laboratorio, el estudiante será capaz de:

1. Comprender la representación de pose como combinación de posición y orientación.
2. Implementar rotaciones usando:
    - Ángulos de Euler
    - Matrices de rotación
    - Cuaterniones
3. Aplicar transformaciones entre sistemas de referencia (frames).
4. Usar cuaterniones en ROS2 para representar orientación.
5. Analizar la no conmutatividad de las rotaciones.
6. Implementar y visualizar transformaciones usando TF2.

🧠 Conceptos clave

- Pose (posición + orientación)
- Transformaciones SE(3)
- Sistemas de referencia (frames)
- Composición de transformaciones
- Cuaterniones
- TF2 en ROS2

📌 Parte 0: Requisitos

Software

- ROS2 (versión compatible con el ubuntu instalado)
- Python 3
- Librerías:

pip install numpy scipy matplotlib

Paquetes ROS2

sudo apt install ros-[version]-tf-transformations

sudo apt install ros-[version]-tf2-ros

🧩 PARTE 1: Representaciones de orientación

🔹 Actividad 1: Conversión de representaciones

Implemente un script en Python que:

1. Genere una rotación de 90° alrededor del eje Z.
2. Obtenga:
    - Matriz de rotación
    - Cuaternión
3. Verifique que ambas representaciones son equivalentes.

Código de prueba:

```
from scipy.spatial.transform import Rotation as R

# Rotación de 90° en Z
r = R.from_euler('z', 90, degrees=True)

q = r.as_quat()   # [x, y, z, w]
R_mat = r.as_matrix()

print("Cuaternión:", q)
print("Matriz:\n", R_mat)
```

📌 Entregable

- Resultados impresos
- Breve explicación (máx. 5 líneas)

🔹 **Actividad 2: No conmutatividad**

1. Defina dos rotaciones:
    - 90° en Z
    - 90° en X
2. Evalúe:
    - $R_1 R_2$
    - $R_2 R_1$

📌 **Pregunta**

¿Los resultados son iguales? Justifique.

Código de prueba (hay que incluir la librería):

```
r1 = R.from_euler('z', 90, degrees=True)
r2 = R.from_euler('x', 90, degrees=True)

r_total = r1 * r2   # composición

print(r_total.as_matrix())
```

🤖 **PARTE 2: ROS2 – Publicación de pose**

🔹 **Actividad 3: Nodo publicador**

Desarrolle un nodo ROS2 que:

- Publique en el tópico /pose
- Incluya:
    - Posición fija
    - Orientación usando cuaterniones

Después de crear el espacio de trabajo y el paquete de prueba, incluya el código publicador en Python:

📄 **archivo: pose_publisher.py**

```
import rclpy
from rclpy.node import Node
from geometry_msgs.msg import Pose
from scipy.spatial.transform import Rotation as R
```

```
class PosePublisher(Node):

    def __init__(self):
        super().__init__('pose_publisher')
        self.pub = self.create_publisher(Pose, 'pose', 10)
        self.timer = self.create_timer(0.5, self.publish_pose)

    def publish_pose(self):
        msg = Pose()

        # posición
        msg.position.x = 1.0
        msg.position.y = 2.0
        msg.position.z = 0.0

        # orientación (90° en Z)
        r = R.from_euler('z', 90, degrees=True)
        q = r.as_quat()

        msg.orientation.x = q[0]
        msg.orientation.y = q[1]
        msg.orientation.z = q[2]
        msg.orientation.w = q[3]

        self.pub.publish(msg)

def main():
    rclpy.init()
    node = PosePublisher()
    rclpy.spin(node)
    node.destroy_node()
    rclpy.shutdown()
```

📌 **Entregable**

- Comandos de ejecución
- Evidencia (terminal o captura)

🔄 **PARTE 3: Transformaciones con TF2**

🔹 **Actividad 4: Árbol de transformaciones**

1. Cree un nodo que publique una transformación:
    - world → robot

2. El robot debe:
    - Rotar continuamente sobre Z

📌 **Visualización**

- Use RViz o view_frames

📌 **Entregable**

- Captura de RViz mostrando frames

Incluya el código publicador en Python y use Rviz para la visualización:

📄 **archivo: tf_broadcaster.py**

```
import rclpy
from rclpy.node import Node
from geometry_msgs.msg import TransformStamped
from tf2_ros import TransformBroadcaster
from scipy.spatial.transform import Rotation as R

class TFPublisher(Node):

    def __init__(self):
        super().__init__('tf_publisher')
        self.br = TransformBroadcaster(self)
        self.timer = self.create_timer(0.1, self.broadcast_tf)
        self.t = 0

    def broadcast_tf(self):
        t = TransformStamped()

        t.header.stamp = self.get_clock().now().to_msg()
        t.header.frame_id = 'world'
        t.child_frame_id = 'robot'

        # Movimiento circular
        t.transform.translation.x = 2.0
        t.transform.translation.y = 0.0
        t.transform.translation.z = 0.0

        # Rotación progresiva
        r = R.from_euler('z', self.t)
        q = r.as_quat()

        t.transform.rotation.x = q[0]
        t.transform.rotation.y = q[1]
        t.transform.rotation.z = q[2]
        t.transform.rotation.w = q[3]
```

```
        self.br.sendTransform(t)
        self.t += 0.05

def main():
    rclpy.init()
    node = TFPublisher()
    rclpy.spin(node)
    node.destroy_node()
    rclpy.shutdown()
```

🔍 **Visualización**

1. ros2 run tf2_tools view_frames
2. o en RViz: rviz2
   Agregar: TF y Axes

🔬 **PARTE 4: Error de orientación**

🔹 **Actividad 5: Cálculo de error**

1. Defina:
   - Orientación deseada: 90° en Z
   - Orientación actual: 45° en Z
2. Calcule el error usando cuaterniones.

Código de prueba:

```
from scipy.spatial.transform import Rotation as R

q_des = R.from_euler('z', 90, degrees=True)
q_act = R.from_euler('z', 45, degrees=True)

q_error = q_des * q_act.inv()

print(q_error.as_euler('xyz', degrees=True))
```

🎯 Interpretación

- Error angular permite implementar control de orientación, buscando que el error tienda a cero en un tiempo determinado.

📌 **Pregunta**

¿Qué representa físicamente el resultado?

🚀 PARTE 5: Interpolación de orientación

🔹 Actividad 6: SLERP

1. Interpole entre:
    - 0° y 90° en Z
2. Genere al menos 5 puntos intermedios.

Código de prueba:

```
from scipy.spatial.transform import Rotation as R, Slerp
import numpy as np

key_times = [0, 1]
rots = R.from_euler('z', [0, 90], degrees=True)

slerp = Slerp(key_times, rots)

times = np.linspace(0, 1, 5)
interp = slerp(times)

print(interp.as_euler('xyz', degrees=True))
```

📌 Entregable

- Resultados
- Interpretación

🧠 DISCUSIÓN FINAL

Responda:

1. ¿Por qué ROS2 usa cuaterniones en lugar de ángulos de Euler?
2. ¿Qué problema evitan los cuaterniones?
3. ¿Qué relación existe entre:
    - TF2
    - Transformaciones del capítulo 2?